Question 2: Understand Testbenches

Testbenches are useful, particularly for unit tests. Test your understanding of writing a testbench using `fir_tb.sv`. Feel free to search for anwers online.

a. How does the `initial` block work?

   It executes once at the beginning of the simulation. It is used to start and set up the testing scenario.

b. How come we didn't use an `initial` block in the FIR to force the signals to a valid state? (Hint: what happens to a module with `initial` blocks during synthesis?)

   The condition in the module is not validated. The testing may run in the wrong condition. We should be suspicious of the synthesis tools like the lecture says

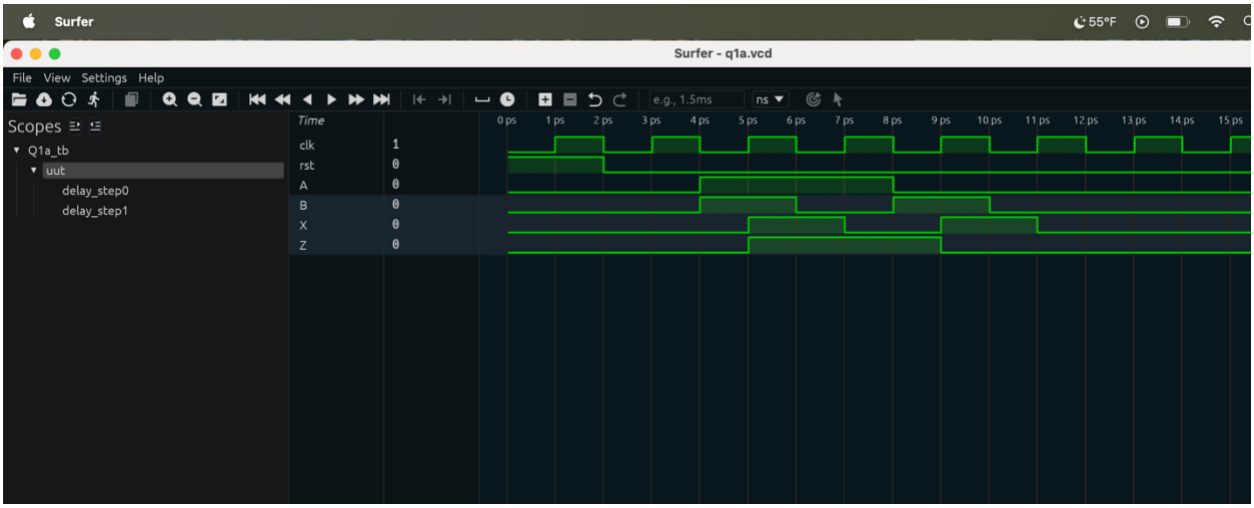
c. Register `In` is driven on the negative edge of the clock. Why not the positive edge? Would this cause a violation? If so, what type?
   In is driven on the negedge clk so that it is ready at the posedge. If it is at posedge, it is called in two places. However, for a non-blocking In, it can only be in one LHS. So, non-blocking type causes the violation.

d. In the line that generates the simulated clock, a special operator `#` is used. What is this operator?
   # is a time delay operator. It asks the simulator to wait a specified amount of time. Like wait for a ½ cycle of clk which means ask the simulator to start at negedge.

e. Is the line generating the clock a continuous assign statement?
   no, that's non-continuous assignment, like sequential logic

Question 3: Writing a Testbench

Submit your SystemVerilog testbench and a screenshot of the simulation waveforms showing all of the input and output signals.



Question 4: Evaluate Gate Level Simulations

Correlate the SDF-annotated timing to the waveform from the gate level simulation.

1. `delay_step0` is the first flip-flop in a chain. What is the output port name of this flip-flop, and how wide is this port?
   output port name is q. Width of this port is 4 bits.
2. Open the `vcdplus.fsdb` file (which should be from the gate level simulation) in Verdi and examine the output of the flip-flop at around 50ns. Calculate the delay of the output transition relative to the input clock's rising edge.
3. Can you correlate this with the delay in the SDF file by identifying the corresponding delay in the SDF file?
   In this instance "(INSTANCE delay_step0.q_reg\[0\])", the output delay in the rising clock edge is 453 ps.
   Delay is from this row: "(IOPATH CK Q (::453) (::491))"

I cannot connect to Berkeley EECS's server and don't have SDF-annotated simulation, so have to skip question 5 and question 6.

Question 5: SDF and Visualization of Setup Time Violations

Examine the non-zero delays in timing-annotated simulations. Clock frequency selection must consider gate delay. If the frequency is too high, there will be setup timing

violations. Edit `sim-gl-syn.yml` to lower the clock period to 5ns (`CLOCK PERIOD=5.00`). Now, simulate again.

1. Does the hardware continue to function correctly?
2. What is the shortest clock period where the FIR still functions correctly (meets timing)?
    - Note: Instead of closing and reopening Verdi for every change, simply reload your design. Click **File** on the top menu bar, then click **Reload Design** to reflect the new configuration after each simulation run.

Revert the clock period back to 20ns and replace the value for the `sdf_file` key in `sim-gl-syn.yml` with `fir.mapped_hold.sdf` (i.e., `sdf_file: fir.mapped_hold.sdf`). This will run the simulation with a different delay file that intentionally has an error. Re-run the gate level simulation. You should now have two different fsdb files (`vcdplus.fsdb` and `vcdplus_orig.fsdb`), which you can load in two seperate Verdi windows to see the differences. Add all of the `delay_chain` signals in the DUT module for both windows. Zoom into the clock edges near 52ns into the simulation. There should be a significant difference between the two fsdb files, and one of them will have a signal that is incorrectly getting captured on the wrong cycle. This is an exaggerated case of a hold time violation, which occurs when one delay path is too small relative to another. **Submit a screenshot.**

Question 6: Fixing Hold Times

Setup time violations can be fixed by increasing the clock period (lowering the frequency), but hold-time violations cannot, because the capturing edge relationship in the hold-time violation does not change with clock period. Later, you will learn how the CAD tools do this for you, but in this problem you will manually identify the error in the SDF and fix it.

a. Explain the differences between the waveforms in the two `.fsdb` files. Which signal(s) are different and why? Show what in the SDF is causing this.
b. Modify the `src/post-syn/fir.mapped_hold.sdf` file to fix the hold time **without changing what you found in (6a). You should not be changing any values to 0**. While you may change parameters for multiple registers, you must change only one parameter PER register. Submit the following:

i. Which delay did you change (show the original and fixed value), and which register(s) did this delay belong to?
ii. Why did you change that register's delay?
iii. Simulation waveforms showing correct output, along with the text printout of the simulation showing that the results are correct.
iv. Since you cannot directly modify `.sdf` files to fix hold-time violations in real designs, give your best guess as to what would be inserted or removed from a gate level implementation of this design to achieve the same hold-time fix.

Question 7: Analyzing Power Reports

1. Open `build/power-rundir/activePowerReports/ss_100C_1v60.setup_view.rpt`. What is the most obvious difference in the power numbers compared to the hold view file? What do you think is the dominant factor contributing to this difference?

   Switching power between hold and setup has the most obvious difference in power. Switching power in hold status is higher than that of setup. I think the dominant factor contributing to this is that based on the equation $P \propto CV^2f$, the hold status consumes more switching voltage.

2. Compare the reports with SAIF annotation you just viewed against the analogous reports in `build/power-rundir/staticPowerReports`. What would you estimate is the effective switching activity factor of the testbench we have been using?

   In hold view, total switching power in active status is 3.427 mW, and total switching power in static status is 3.664 mW. From the instruction, we get "static power analysis by default assumes an average switching activity factor of 0.2". By using the equation $P \propto CV^2f$, we get:
   P(active switching) / ∝(active switching) = P(static switching) / ∝(static switching)
   3.427 mW / ∝(active switching) = 3.664 mW / 0.2
   ∝(active switching) = 0.187

   Note: Switching power is linearly proportional to switching activity.